

RustChef - Technical Report

Repository: <https://github.com/MedElMostapha/RustChef>

Overview

RustChef is a command-line tool inspired by [CyberChef](#), the "Cyber Swiss Army Knife". It provides 16 data transformation, encoding, decoding, hashing, extraction, and analysis operations from the terminal. The tool is written in Rust and follows a modular architecture with a clean CLI interface.

Architecture

RustChef follows a single-binary, modular architecture:

```
src/
|-- main.rs           Entry point: CLI parsing, input reading, operation dispatch
|-- cli.rs            Clap derive definitions for all 16 subcommands
|-- ops/
|   |-- mod.rs         Module declarations
|   |-- base64_op.rs   Base64 encode / decode
|   |-- hex_op.rs      Hex encode / decode
|   |-- url_op.rs      URL percent encode / decode
|   |-- hash_op.rs     MD5, SHA1, SHA256 hashing
|   |-- rot13_op.rs    ROT13 letter substitution
|   |-- xor_op.rs      XOR with repeating byte key
|   |-- extract_op.rs  IP, URL, email regex extraction
|   |-- stats_op.rs    Text statistics (char, word, line, byte, counts)
|   |-- entropy_op.rs  Shannon entropy (bits/byte)
```

Control Flow

1. `main.rs` parses CLI arguments via `clap` (derive API)
2. Depending on the subcommand, `main.rs` calls the appropriate `ops::*` function
3. Input is read either from a positional argument or from stdin (if no argument is given)
4. Each operation returns a `Result<String>`; errors are propagated via `anyhow`
5. Output is printed to stdout via `println!`

Design Decisions

Decision	Rationale
Single subcommand per invocation	Follows Unix philosophy of composability; chaining via <code>\ </code>
Positional argument or stdin	Enables both direct use and pipe-based use
Binary output as hex fallback	When decode/XOR output is not valid UTF-8, hex representation is shown
anyhow for errors	Simple, ergonomic error propagation with clear user messages
Clap derive API	Type-safe, concise, auto-generated <code>--help</code>
No streaming	Simpler implementation; acceptable for a CLI tool processing typical inputs

Implemented Operations

#	Subcommand	Category	Description
1	<code>base64-encode</code>	Encoding	Encode bytes to Base64 (RFC 4648)
2	<code>base64-decode</code>	Encoding	Decode Base64 to bytes
3	<code>hex-encode</code>	Encoding	Encode bytes to lowercase hex string
4	<code>hex-decode</code>	Encoding	Decode hex string to bytes
5	<code>url-encode</code>	Encoding	Percent-encode a string (RFC 3986)
6	<code>url-decode</code>	Encoding	Percent-decode a string
7	<code>rot13</code>	Transformation	Apply ROT13 cipher (a->n, b->o, etc.)
8	<code>xor</code>	Transformation	XOR bytes with a repeating key
9	<code>md5</code>	Hashing	Compute MD5 digest (128-bit, hex)
10	<code>sha1</code>	Hashing	Compute SHA1 digest (160-bit, hex)
11	<code>sha256</code>	Hashing	Compute SHA256 digest (256-bit, hex)
12	<code>extract-ips</code>	Extraction	Extract IPv4 addresses (dotted decimal) and IPv6 addresses

#	Subcommand	Category	Description
13	<code>extract-urls</code>	Extraction	Extract HTTP/HTTPS/FTP URLs
14	<code>extract-emails</code>	Extraction	Extract email addresses
15	<code>stats</code>	Analysis	Count characters, words, lines, bytes, letters, digits, whitespace, punctuation
16	<code>entropy</code>	Analysis	Compute Shannon entropy in bits per byte

Dependencies

Crate	Version	Purpose
<code>clap</code>	4	CLI argument parsing with derive API
<code>base64</code>	0.22	Base64 encoding/decoding (general purpose)
<code>percent-encoding</code>	2	URL percent encoding/decoding
<code>sha2</code>	0.10	SHA256 cryptographic hash
<code>sha1</code>	0.10	SHA1 cryptographic hash
<code>md-5</code>	0.10	MD5 cryptographic hash
<code>regex</code>	1	Regex-based IP, URL, email extraction
<code>anyhow</code>	1	Simple error handling and context

Testing Strategy

Test Suite (30 integration tests)

All tests are in `tests/integration.rs` and invoke the compiled binary via `env!` (`"CARGO_BIN_EXE_rustchef"`).

Coverage categories:

Category	Tests	Examples
Known-answer correctness	12	<code>base64-encode "hello" -> "aGVsbG8="</code> , <code>md5 "hello" -> "5d41402abc4b2a76b9719d911017c592"</code>
Roundtrip fidelity	6	encode -> decode returns original for Base64, hex, URL, ROT13, XOR

Category	Tests	Examples
Error handling	5	invalid Base64, invalid hex, odd-length hex, empty XOR key, invalid URL encoding
Stdin input	1	pipe input to <code>base64-encode</code>
Edge cases	3	empty input for stats, ROT13 on numbers, entropy of uniform data
Extraction	3	IP addresses (including invalid ones), URLs, emails

Running Tests

```
cargo test          # all 30 integration tests
cargo clippy        # zero warnings
```

Evidence of Execution

1. Help message

```
med-lemine@client:~/SupNum/RustChef$ rustchef --help
A CyberChef-inspired CLI tool for data transformations

Usage: rustchef <COMMAND>

Commands:
  base64-encode  Encode data to Base64
  base64-decode  Decode Base64 data
  hex-encode     Encode data to hexadecimal
  hex-decode    Decode hexadecimal data
  url-encode     URL-encode a string
  url-decode    URL-decode a string
  rot13         Apply ROT13 transformation
  xor           XOR data with a key
  md5           Compute MD5 hash
  sha1          Compute SHA1 hash
  sha256        Compute SHA256 hash
  extract-ips    Extract IPv4 and IPv6 addresses
  extract-urls   Extract URLs from text
  extract-emails Extract email addresses
  stats         Compute text statistics
  entropy        Compute Shannon entropy
  help          Print this message or the help of the given subcommand(s)

Options:
  -h, --help      Print help
  -V, --version   Print version
med-lemine@client:~/SupNum/RustChef$
```

2. All 30 tests passing

```
med-lemine@client:~/SupNum/RustChef$ cargo test
Finished `test` profile [unoptimized + debuginfo] target(s) in 0.13s
Running unittests src/main.rs (target/debug/deps/rustchef-bc8df1dbf270bf1c)

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

Running tests/integration.rs (target/debug/deps/integration-9f7a78019da5c267)

running 30 tests
test test_empty_stats_again ... ok
test test_empty_stats ... ok
test test_base64_encode ... ok
test test_base64_decode ... ok
test test_entropy_high ... ok
test test_entropy ... ok
test test_base64_decode_invalid ... ok
test test_base64_roundtrip ... ok
test test_hex_decode ... ok
test test_hex_decode_invalid ... ok
test test_hex_encode ... ok
test test_hex_decode_odd_length ... ok
test test_md5 ... ok
test test_rot13_numbers ... ok
test test_rot13 ... ok
test test_hex_roundtrip ... ok
test test_extract_emails ... ok
test test_stats ... ok
test test_sha256 ... ok
test test_rot13_roundtrip ... ok
test test_sha1 ... ok
test test_url_encode ... ok
test test_url_decode ... ok
test test_stdin_input ... ok
test test_xor ... ok
test test_xor_empty_key ... ok
test test_extract_urls ... ok
test test_url_roundtrip ... ok
test test_xor_roundtrip ... ok
test test_extract_ips ... ok

test result: ok. 30 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.04s

med-lemine@client:~/SupNum/RustChef$
```

3. Clippy with zero warnings

```
med-lemine@client:~/SupNum/RustChef$ cargo clippy -- -D warnings
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.12s
med-lemine@client:~/SupNum/RustChef$
```

4. Encoding / Decoding (Base64, Hex, URL, ROT13)

```
med-lemine@client:~/SupNum/RustChef$ echo "=== Base64 ===" && rustchef base64-encode "CyberChef" && rustchef base64-decode "Q3liZXJDVGVm" && echo "=== Hex ===" && rustchef hex-encode "hello" && rustchef hex-decode "68656c6c6f" && echo "=== URL ===" && rustchef url-encode "a b&c=d?e/f" && rustchef url-decode "a%20b%26c%3Dd%3Fe%2Ff"
=== Base64 ===
Q3liZXJDVGVm
CyberChef
=== Hex ===
68656c6c6f
hello
=== URL ===
a%20b%26c%3Dd%3Fe%2Ff
a b&c=d?e/f
med-lemine@client:~/SupNum/RustChef$
```

5. Hashing (MD5, SHA1, SHA256)

```
med-lemine@client:~/SupNum/RustChef$ rustchef md5 "medlemine" && rustchef sha1 "medlemine" && rustchef sha256 "medlemine"
41e5978569a2d5abaa2fff1d81bc692c
768a1ffb75b2b2a9ec2d6d9f26f263ba73c36912
9f9a664c089f8256ec25050b8466684b11d9339d3d47c6ad93be8199a5d55a14
med-lemine@client:~/SupNum/RustChef$
```

6. ROT13 and XOR

```
med-lemine@client:~/SupNum/RustChef$ echo "=== ROT13 ===" && rustchef rot13 "Hello, World!" && rustchef rot13 "Uryyb, Jbeyq!" && echo "=== XOR ===" && rustchef xor --key secret "my data" | rustchef xor --key secret
=== ROT13 ===
Uryyb, Jbeyq!
Hello, World!
=== XOR ===
my data
med-lemine@client:~/SupNum/RustChef$
```

7. Extraction (IPs, URLs, Emails)

```
med-lemine@client:~/SupNum/RustChef$ rustchef extract-ips "Server: 192.168.1.1, gateway: 10.0.0.1" && rustchef extract-urls "Visit https://example.com and http://test.org" && rustchef extract-emails "Contact: user@example.com or admin@test.org"
10.0.0.1
192.168.1.1
http://test.org
https://example.com
admin@test.org
user@example.com
med-lemine@client:~/SupNum/RustChef$
```

8. Analysis (Stats, Entropy)

```
med-lemine@client:~/SupNum/RustChef$ rustchef stats "Hello World! This is a test." && rustchef entropy "abcdefghijklm
nop"
Character count: 28
Word count:      6
Line count:      1
Byte count:      28
Letter count:    21
Digit count:     0
Whitespace count: 5
Punctuation count: 2
4
med-lemine@client:~/SupNum/RustChef$
```

9. Piped stdin input

```
med-lemine@client:~/SupNum/RustChef$ echo "hello from stdin" | rustchef base64-encode
aGVsbG8gZnJvbSBzdGRpbG==
med-lemine@client:~/SupNum/RustChef$
```

10. Docker run

```
med-lemine@client:~/SupNum/RustChef$ docker run --rm rustchef sha256 "medlemine"
9f9a664c089f8256ec25050b8466684b11d9339d3d47c6ad93be8199a5d55a14
med-lemine@client:~/SupNum/RustChef$
```


11. Docker build

```
med-lemine@client:~/SupNum/RustChef$ docker build -t rustchef .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   448kB
Step 1/10 : FROM rust:latest AS builder
--> e0b053325bdc
Step 2/10 : WORKDIR /app
--> Using cache
--> 3a7b1c6cdb6d
Step 3/10 : COPY Cargo.toml Cargo.lock ./
--> Using cache
--> e1aa23301dbe
Step 4/10 : COPY src ./src
--> Using cache
--> f6ae9f977a05
Step 5/10 : RUN cargo build --release
--> Using cache
--> 953426022002
Step 6/10 : FROM debian:bookworm-slim AS runtime
--> 1f2403136cb0
Step 7/10 : RUN apt-get update && apt-get install -y --no-install-recommends ca-certificates && rm -rf /var/lib/apt/lists/*
--> Using cache
--> 887f8353ff63
Step 8/10 : COPY --from=builder /app/target/release/rustchef /usr/local/bin/rustchef
--> Using cache
--> d31b112084c0
Step 9/10 : ENTRYPOINT ["rustchef"]
--> Using cache
--> a3de8653eb02
Step 10/10 : CMD ["--help"]
--> Using cache
--> 623a76f75803
Successfully built 623a76f75803
Successfully tagged rustchef:latest
med-lemine@client:~/SupNum/RustChef$
```

12. Sample file processing

```
med-lemine@client:~/SupNum/RustChef$ cat samples/input.txt | rustchef extract-ips && echo "---" && cat samples/input.txt | rustchef stats
10.0.0.50
172.16.0.10
192.168.1.100
2001:0db8:85a3:0000:0000:8a2e:0370:7334
8.8.8.8
---
Character count: 767
Word count: 93
Line count: 27
Byte count: 767
Letter count: 471
Digit count: 80
Whitespace count: 97
Punctuation count: 119
med-lemine@client:~/SupNum/RustChef$
```

Limitations

1. **No recipe chaining:** Unlike CyberChef, operations cannot be chained within a single invocation. Users must pipe commands or use intermediate files.

2. **Binary output as hex:** Non-UTF-8 binary output is displayed as a hex string rather than raw bytes, limiting use in binary pipelines.
3. **Extraction regexes:** Regex-based extraction may miss edge cases (IPv6 compressed notation, unusual URL schemes, internationalized email addresses).
4. **In-memory processing:** The tool reads the entire input into memory, which may be problematic for very large files (>1GB).
5. **Single Rust edition:** Requires Rust edition 2021; not compatible with older toolchains.

Deliverables

Item	Location
GitHub repository	https://github.com/MedElMostapha/RustChef
Rust source code	<code>src/</code>
Cargo manifest	<code>Cargo.toml</code> , <code>Cargo.lock</code>
Integration tests	<code>tests/integration.rs</code>
Dockerfile	<code>Dockerfile</code>
README	<code>README.md</code>
Technical report	<code>docs/report.md</code>
Sample input	<code>samples/input.txt</code>
Expected outputs	<code>samples/expected/</code>